PANEL PREPARATION GUIDE

Paper Explanation, Reframed PD & Objectives

Complete reference for defending your TGDetect project in front of the panel, based on the Pomsathit (2025) IEEE Access paper

REFERENCE PAPER	JOURNAL	FOCUS
Pomsathit 2025	IEEE Access	GNN + Continual Learning

Contains: Full paper explanation, what to say if panel asks about it, how it compares to TGDetect, the reframed Problem Definition for your poster, and 4 polished objectives.

SECTION 1

Paper Overview - What This Paper Is About

Full Citation

Pomsathit, A. (2025). "Adaptive Detection of Advanced Persistent Threats (APT) With Graph Neural Networks and Rehearsal-Based Continual Learning on Wazuh EDR Telemetry." **IEEE Access**, Volume 13, DOI: 10.1109/ACCESS.2025.3639270. Open Access.

This paper proposes an adaptive APT detection framework that combines **Graph Neural Networks (GNNs)** with **rehearsal-based continual learning** using real endpoint telemetry from **Wazuh**, an open-source SIEM/EDR platform. The author is Auttapon Pomsathit from King Mongkut's Institute of Technology Ladkrabang, Thailand. The paper was published in IEEE Access in December 2025 and is **fully open access**.

What Problem Does This Paper Solve?

The paper addresses two fundamental problems in APT detection. First, **concept drift**: attack behaviors evolve over time, so a model trained on yesterday's attacks may fail to detect tomorrow's. Second, **catastrophic forgetting**: when you update a model with new attack data, it tends to "forget" what it learned about older attack patterns. The paper solves both by using rehearsal-based continual learning, which stores a small buffer of past samples and replays them during training so the model retains old knowledge while learning new patterns.

What Data / Logs Does This Paper Use?

This is critical for your panel defense. The paper explicitly uses **Wazuh EDR telemetry**, which includes four specific log types: **(1) Process creation/termination logs** (tracking malicious script execution), **(2) File operation logs** (creation, modification, deletion), **(3) Registry access logs** (for detecting persistence mechanisms), and **(4) Network connection logs** (for lateral movement and command-and-control communication). The benign data was collected from Wazuh agents deployed in an enterprise environment capturing normal activities like office application usage and web browsing. The malicious data was generated by executing APT samples in a controlled sandbox following the **MITRE ATT&CK framework**.

KEY FOR PANEL

When asked "What logs does this paper use?", say: "Pomsathit (2025) uses Wazuh EDR telemetry covering four specific log types: process creation/termination logs, file operation logs, registry access logs, and network connection logs. Both benign traces from enterprise Wazuh agents and malicious traces from sandbox-executed APT scenarios following MITRE ATT&CK were used."

SECTION 2

Algorithm & Methodology

The paper's pipeline has four stages. Understanding each stage is essential for panel defense:

Stage 1: Data Collection with Wazuh EDR

Wazuh agents monitor endpoints and collect telemetry about process creation/termination, file operations, registry access, and network connections. The dataset combines approximately **12,500 benign graphs** (1.2 million nodes, 3.5 million edges) from enterprise telemetry with **8,300 malicious graphs** (0.9 million nodes, 2.7 million edges) from sandbox-executed APT scenarios.

Stage 2: Graph Construction

Telemetry is represented as a graph G = (V, E, X) where V is the set of nodes (processes, files, IPs, registry keys), E is the set of edges (interactions between nodes), and X is the node feature matrix containing attributes such as command lines, file hashes, and IP addresses. This graph-based representation captures **multi-hop dependencies** across attack stages - for example, a privilege escalation followed by lateral movement appears as a connected path in the graph, which a flat table of logs would not reveal.

Stage 3: GNN-Based Detection

A **multi-layer GNN** (3 layers, hidden dimension = 128, ReLU activation, dropout = 0.2) propagates information across graph neighborhoods. Each node's representation is updated by aggregating features from its neighbors, applying a transformation weight matrix, and passing through a non-linear activation function. At the output layer, a softmax function classifies each node as benign or malicious. The aggregation function used is **attention-based**, meaning the model learns to weight important neighbors more heavily.

Stage 4: Rehearsal-Based Continual Learning

This is the paper's key innovation. The total loss at each training step is: **L = L_new(D_t) + lambda * L_rehearsal(D_buf)**. Here, L_new is the loss on the current batch of new data, L_rehearsal is the loss on a small buffer of past samples (10% of historical data, updated via reservoir sampling), and lambda is a balancing hyperparameter. This dual-loss approach means the model simultaneously learns new attack patterns AND retains knowledge of old ones, solving both concept drift and catastrophic forgetting.

The GNN formula (Equation 3): h_v^(l+1) = sigma(W^(l) * AGGREGATE(h_v^(l) UNION h_u^(l) for u in N(v))). In plain English: each node collects information from all its neighbors, combines it with its own current knowledge, transforms it through a learned weight matrix, and applies an activation function. After multiple layers, each node has accumulated context from its entire neighborhood, enabling it to detect suspicious patterns based on both local features and relational context.

SECTION 3

Key Results You Must Know

The panel may ask what this paper achieved. Memorize these specific numbers:

Metric	Proposed Model (GNN + Rehearsal)	Static GNN	Fine-Tuned GNN	CNN-BiLSTM
F1-Score	**0.981**	0.931	0.960	0.931
Recall	**0.978**	0.928	0.952	0.929
AUC	**0.984**	0.941	0.965	0.938
Precision	**0.985**	0.935	0.968	0.933

The proposed model achieved **F1-scores above 0.98** across evolving APT scenarios, improving by **+5.0%** compared to CNN-BiLSTM and **+2.1%** compared to fine-tuned GNN. The detection latency overhead was only **approximately 2 ms** compared to static GNNs, and the rehearsal buffer required only **10% additional storage**. These numbers prove that continual learning adds minimal overhead while significantly improving detection accuracy under evolving threat conditions.

Performance by MITRE ATT&CK Tactic

The paper also evaluated detection performance across specific attack tactics. For **persistence** and **privilege escalation**, the model achieved F1-scores above 0.98, because these tactics leave clear graph signatures (file modifications, process injections) that the GNN captures well. For **command-and-control (C2)** detection, the F1-score was lower at **0.936**, because C2 communication is stealthy and often uses encrypted traffic, making it harder to detect. The paper acknowledges this as a limitation and an area for future research.

Ablation Study Insights

The ablation study (Table 5 in the paper) proves that **both components are necessary**. GNN alone without continual learning achieved relatively high accuracy but could not adapt to evolving attacks. Rehearsal-based continual learning applied to sequential models improved retention but underperformed in recall and F1-score because sequential representations fail to capture relational context. Fine-tuned GNNs achieved modest improvements but suffered from catastrophic forgetting. Only the **full integration** of GNN + rehearsal achieved the highest F1-score of 0.981, confirming a synergistic effect.

SECTION 4

How This Paper Relates to TGDetect

If the panel asks "How does your project relate to this paper?" or "Are you just implementing what this paper did?", use the comparison below. This paper covers **one piece** of what TGDetect proposes; your project is broader in scope.

Pomsathit (2025) vs. TGDetect - Key Differences

POMSATHIT PAPER

- Uses ONLY Wazuh EDR logs (1 source) - Uses a standard GNN (not temporal) - Focus: continual learning with rehearsal buffer - Detects APTs only - No explainability module - No attack backtracking - Static graph snapshots (no time dimension)

TGDETECT (YOUR PROJECT)

- Fuses 5 log sources (auth, audit, network, DNS, cloud) - Uses a Temporal GNN (time-aware) - Focus: temporal dynamics + drift adaptation - Detects APTs + LotL + Zero-day - Has temporal explainability module - Has attack backtracking engine - Continuous temporal graph with time-evolving edges

If Panel Asks: "Are You Just Reproducing This Paper?"

"No. Pomsathit (2025) uses a standard GNN with Wazuh EDR logs only, focusing on rehearsal-based continual learning. TGDetect goes significantly further in four ways. First, we fuse five heterogeneous log sources into a unified temporal graph, not just one EDR source. Second, we use a Temporal GNN that explicitly captures time-evolving attack dynamics, not static graph snapshots. Third, we incorporate temporal explainability and attack backtracking, which this paper does not address. Fourth, we address three attack categories - APTs, Living-off-the-Land, and zero-day threats - with a framework that maps detections to the MITRE ATT&CK framework. Our work builds upon the graph-based approach validated in this paper but extends it substantially."

If Panel Asks: "What Did You Learn From This Paper?"

"This paper validated several design decisions for TGDetect. First, it confirmed that graph-based representations outperform sequence-based models for multi-stage attack detection, achieving F1-scores above 0.98 compared to 0.931 for CNN-BiLSTM. Second, it demonstrated that rehearsal-based continual learning is an effective strategy for handling concept drift, which directly supports our online concept drift adaptation module. Third, it showed that multi-hop graph dependencies are crucial for detecting lateral movement and privilege escalation patterns. However, it also revealed gaps that TGDetect addresses: the lack of temporal awareness in the graph, the reliance on a single log source, and the absence of explainability for analyst decision-making."

SECTION 5

Panel Q&A Quick Reference

Q: "What is concept drift and why does it matter?"

"Concept drift refers to the phenomenon where attack behaviors evolve over time. A model trained on today's attack patterns may become less effective as attackers adopt new techniques. In the Pomsathit paper, this is demonstrated by showing that static GNNs degrade in performance when evaluated on evolving APT scenarios. Their solution - rehearsal-based continual learning - stores a buffer of past samples and replays them during training. TGDetect addresses the same problem through our online concept drift adaptation module, but we extend it with temporal awareness - we not only maintain detection accuracy as patterns shift, but we also track WHEN patterns change, providing temporal context that the Pomsathit paper does not capture."

Q: "What is catastrophic forgetting?"

"Catastrophic forgetting occurs when a neural network is updated with new data and inadvertently overwrites previously learned knowledge. In the APT detection context, this means if you retrain your model to detect a new attack variant, it may forget how to detect older variants. The Pomsathit paper quantifies this: their fine-tuned GNN achieves an F1-score of only 0.960 because updating on new data degrades performance on old data. Their rehearsal-based approach solves this by maintaining a buffer of historical samples. In TGDetect, our entity memory mechanism serves a similar purpose but operates at the node level, allowing individual entities to maintain their behavioral history rather than relying on a global replay buffer."

Q: "What is rehearsal-based continual learning?"

"Rehearsal-based continual learning is a strategy where, during model training, you not only train on new data but also 'rehearse' a small subset of past data. The loss function combines two components: the loss on the current batch and the loss on a rehearsal buffer of past samples. In Pomsathit's paper, the buffer size is 10% of historical data, updated via reservoir sampling. This ensures the model retains knowledge of old attack patterns while adapting to new ones. The total loss is L = L_new + lambda * L_rehearsal, where lambda controls the trade-off between learning new information and retaining old knowledge."

Q: "Why use graphs instead of traditional log analysis?"

"Traditional log analysis treats each event independently, but multi-stage attacks like APTs unfold as connected sequences of actions across multiple entities. Pomsathit's paper demonstrates this empirically: their GNN-based approach achieves an F1-score of 0.981 by capturing multi-hop dependencies across processes, files, and network entities, while a sequence-based CNN-BiLSTM model achieves only 0.931 because it cannot model the relational structure of attacks. A graph naturally represents the attack chain: user logs into host, host spawns process, process accesses file, process connects to external IP. This connected path IS the attack, and only a graph-based model can see it."

Q: "What is Wazuh and why did they use it?"

"Wazuh is an open-source SIEM and EDR platform that monitors endpoints and collects telemetry. The paper chose Wazuh because it provides comprehensive, real-world endpoint telemetry in a reproducible manner, unlike simulated logs such as Windows Sysmon which don't fully capture real SOC environments. Wazuh agents specifically monitor: process creation/termination, file operations, registry access, and network connections. These four log types are the ones explicitly mentioned in the paper as the data sources for graph construction."

SECTION 6

Reframed Problem Definition (For Poster)

The panel criticized the original PD for: starting unprofessionally ("Current...", "nowadays"), lacking impact in the opening lines, not properly integrating the three attack types, and failing to clearly separate the problem from the solution. Below is the reframed version. It is shorter to fit a poster while maintaining authority and impact.

REFRAMED PROBLEM DEFINITION - FOR YOUR POSTER

Advanced Persistent Threats, Living-off-the-Land techniques, and zero-day exploits share a defining characteristic: they unfold as chains of individually benign events across time. A legitimate login at an unusual hour, a trusted process executing an unknown script, a routine file access followed by an outbound network connection, each harmless in isolation but lethal when connected sequentially.

Conventional security systems analyze log events in isolation, severing the temporal and relational context that binds multi-stage attacks together. The result is fragmented, high-volume alerts that describe individual symptoms without revealing the underlying attack narrative.

TGDetect addresses this by reconstructing system activity as a continuous temporal graph, where entities are nodes, their interactions are timestamped edges, and a Temporal Graph Neural Network learns behavioral patterns across this time-evolving structure, enabling detection, correlation, and reconstruction of multi-step attack chains invisible to event-by-event analysis.

What changed from the original and why: The opening now leads with the three attack types as the foundation (not a list tacked on at the end), creating immediate impact. It paints a vivid attack scenario (login + script + file access + network) so the panel FEELS the problem. The word

"conventional" replaces "current" for a professional tone. The problem-solution structure is crystal clear: Paragraph 1 defines the threat, Paragraph 2 defines the gap, Paragraph 3 defines the solution. No grammar errors, no run-on sentences, active voice throughout.

SECTION 7

Reframed Objectives (4 Major Objectives)

The panel wants objectives that start with "To" and demonstrate clear, measurable, and impactful goals. Each objective below is designed to sound strong, specific, and defensible.

1 **To develop a heterogeneous continuous-time Temporal Graph Neural Network (TGNN) that fuses five distinct log sources into a unified temporal graph for modeling multi-step cyberattack patterns.**

What it means: Build a TGNN that combines authentication logs, system audit logs, network flow logs, DNS query logs, and cloud service logs into a single time-aware graph. The word "heterogeneous" signals that the graph handles multiple entity types (users, hosts, processes, files, IPs). "Continuous-time" means edges carry precise timestamps, not just sequence order.

2 **To incorporate an online concept drift adaptation mechanism that enables the TGNN model to continuously learn and adapt to evolving attack behaviors without degrading detection accuracy on previously learned patterns.**

What it means: Implement a continual learning strategy (inspired by the rehearsal-based approach in the Pomsathit paper) so the model stays accurate as new attack variants emerge. "Online" means the adaptation happens during operation, not through periodic offline retraining. This directly addresses concept drift and catastrophic forgetting.

3 **To design an attack backtracking engine that automatically reconstructs the complete multi-step attack chain from the detected alert to the initial point of compromise, enabling full attack scenario reconstruction for incident response.**

What it means: When the TGNN flags an attack, the backtracking engine traces backward through the temporal graph to find the entry point, follow the chain of compromised entities, and present the full attack narrative. This goes beyond detection - it provides actionable intelligence for the security analyst to respond to the incident.

4 **To provide temporal explainability by mapping detected attack patterns and model attention weights to the MITRE ATT&CK framework, enabling security analysts to understand not only what was detected but when, how, and why each event was classified as malicious.**

> **What it means:** The explainability module translates the TGNN's internal decision-making into human-readable attack narratives mapped to MITRE ATT&CK tactics. It provides temporal context (WHEN each step happened), relational context (HOW entities were connected), and model reasoning (WHY the attention mechanism flagged each event). This bridges the gap between AI detection and analyst decision-making.

Why these 4 objectives work for the panel: They cover the full pipeline from data fusion (Obj 1) to model adaptation (Obj 2) to actionable output (Obj 3) to human interpretability (Obj 4). Each objective is measurable, specific, and maps to a distinct technical component. They avoid vague phrases like "analyze logs" or "use AI" and instead specify exact technologies and mechanisms. If the panel asks "What's your contribution?", these four objectives ARE your contribution.

SECTION 8

Quick Comparison: Pomsathit vs. TGDetect

Use this table if the panel asks for a direct comparison between the referenced paper and your project:

Aspect	Pomsathit (2025)	TGDetect (Your Project)
Log Sources	Wazuh EDR only (1 source)	5 sources: Auth, Audit, Network, DNS, Cloud
Graph Type	Static graph G = (V, E, X)	Temporal graph with time-evolving edges
Neural Network	Standard GNN (3 layers, dim=128)	Temporal GNN with time encoding
Adaptation	Rehearsal buffer (10% of past data)	Online concept drift + entity memory
Attack Types	APTs only	APTs + LotL + Zero-day
Explainability	Subgraph visualization only	Temporal explainability + MITRE mapping
Backtracking	Not addressed	Attack path reconstruction engine
Results	F1 > 0.98	Expected comparable or higher

The bottom line: **Pomsathit validated that graph-based detection with continual learning works.** TGDetect takes that validated foundation and extends it with temporal awareness, multi-source fusion, explainability, and backtracking. Pomsathit proved the concept; TGDetect expands the scope.